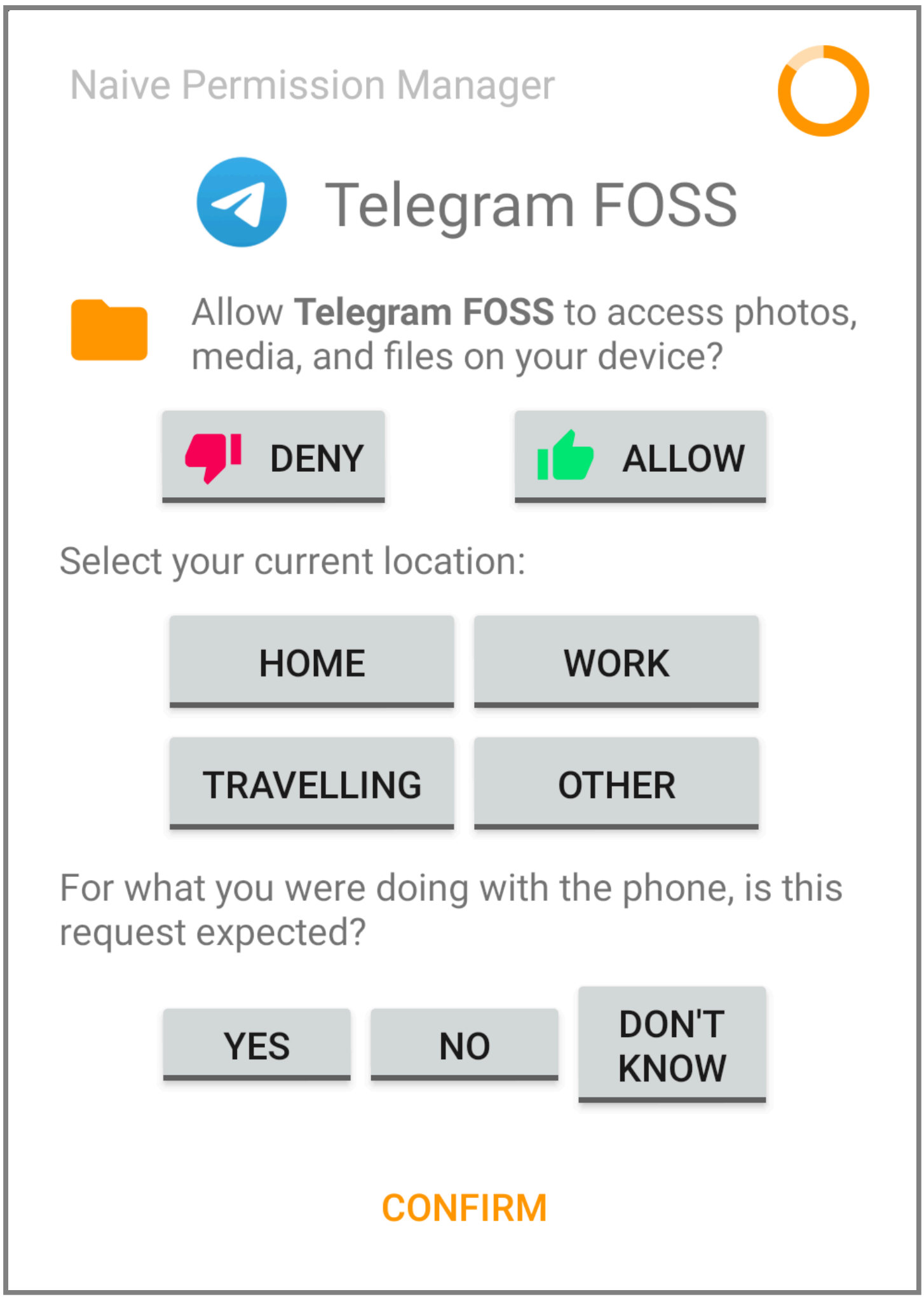
Note: This page is currently only available in english

CM-NPM

The COP-MODE Naive Permission Manager (CM-NPM) is both a permission manager and our data collection tool. Specifically, CM-NPM intercepts permission checks done by any app to the operative system and prompts the user to either accept or deny the permission at that specific time, while collecting contextual data. The answer to a particular permission for a given app is cached for 30 minutes to avoid constant prompts.

CM-NPM is publicly available here.

An anonymized version of the collected dataset is made available to researchers. If you are interested in using this dataset, please reach out to us through cop-mode@dei.uc.pt.



At each permission request, we collect data about:

- **Requesting application:** name and play store category
- **Permission:** name, group and grant result (allow/deny)
- **Phone state:** geolocation, plug, dock, call and screen state, network connection and apps running in the foreground/background

- **User context:** time, semantic location and if the user is in an event, according to their calendar.
- **Expectancy:** answer to “For what you were doing with the phone, is this request expected?”

The previous list is summarized. Full details about the collected data can be found here.

On the left, we have an example of the application "Telegram FOSS" trying to access the "Storage" permission. Note that both the icon of the permission and the text of the permission are equal to the ones used by Android's default permission manager. This was made to ensure that the responses would not be biased by giving extra information.

Implementation Details

To intercept the operative system API calls, we implemented CM-NPM as an EdXposed module. EdXposed is a fork of the Xposed framework to support newer Android versions (up to Android 11). Xposed allows modules to change the behavior of other apps or the system itself without changing any APKs, by adding an additional library (Xposed) to `app_process`, the process that spawns every application. Xposed then allows for modules to hook API calls, including to the OS, and therefore modify the functionally before and/or after the API execution.

With runtime permissions, apps need to check if they have a permission before executing the API call that would collect the data. If the permission is denied (either previously denied by the user, or never requested), the app may or may not request the permission. This conditional execution led us to primarily hook `permission checks` and escalating them to permission requests in the form of the prompts as previously illustrated, while intercepting `permission requests` only to override the result with the participant input given to the prompt created by CM-NPM at the respective permission check. Note that while we collect the data at permission checks, we refer to this data as `permission requests`.

In Android, permissions can have different protection levels, in where only dangerous permissions require to be explicitly requested by the applications. These permissions are considered **dangerous** because they allow apps to access sensitive data or resources that can affect the system and/or the privacy of the user. Therefore, CM-NPM only handles, and therefore collects, permission requests from dangerous permissions. Furthermore, because the default Android permission manager manages the permissions at a group level, a discussable implementation decision, so does CM-NPM. That is, by default, if, for instance, a permission requires the read calendar permission and the user grants it, the app will automatically be granted the write calendar permission on request. While one can argue about this feature from a privacy perspective, CM-NPM follows this behavior as to replicate Android's permission manager from a data collection point of view.

At a dangerous permission check call made by an app, CM-NPM prompts the user as previously illustrated and collects the contextual data as aforementioned. The answer is cached for 30 minutes, thus returning the same answer for the given app and permission group for this duration, in order to avoid warning fatigue. The permission icon and permission description are obtained directly from the Android operating system, so as to not bias the response. To avoid breaking functionality, CM-NPM does not handle permission requests from system apps, letting the Android native permission manager handle those.

The code for CM-NPM is available here. Feel free to use and modify it, but please cite our work. Thank you.

COP-MODE

COP-MODE is a research project funded through the European Union’s Horizon 2020 research and innovation programme under the NGI_TRUST grant agreement no 825618

Contact

cop-mode@dei.uc.pt

© 2019-2020 COP-MODE